



**NMAM INSTITUTE
OF TECHNOLOGY**

Department of Electronics & Communication Engineering

May - 2025

A Project report on

**NUMBER TYPE DETECTION SYSTEM: PRIME,
EVEN, AND ODD CLASSIFICATION USING
SYSTEM VERILOG**

Submitted By:

B N Dikshith

B Pranav Bhasker Nayak

Bhargavi Shetty

Bhavish U R

NNM23EC035

NNM23EC036

NNM23EC037

NNM23EC038



**NMAM INSTITUTE
OF TECHNOLOGY**

Department of Electronics & Communication Engineering

CERTIFICATE

This is to certify that **B N Dikshith** (NNM23EC035), **B Pranav Bhasker Nayak** (NNM23EC036), **Bhargavi Shetty** (NNM23EC037), **Bhavish U R** (NNM23EC038), bonafide students of N.M.A.M. Institute of Technology, Nitte, have submitted a project report entitled “**Number Type Detection System: Prime, Even, and Odd Classification Using System Verilog**” as part of the Project-based SystemVerilog Laboratory, in partial fulfillment of the requirements for the award of Bachelor of Technology Degree in Electronics and Communication Engineering during the year 2024-2025.

Name of the Examiner

Signature with date

ABSTRACT

This paper presents the design and verification of a number type detection system in SystemVerilog, focusing on classifying numbers as prime, even, or odd. The design follows three approaches: Dataflow, Behavioral, and Structural. The Dataflow approach uses continuous assignments and logical operations, while the Behavioral method employs procedural constructs for flexibility. The Structural approach decomposes the design into modular components. These approaches are compared based on efficiency, scalability, readability, and hardware mapping. The project utilizes SystemVerilog's object-oriented features for creating reusable testbenches, simulating the design, and validating the results.

Contents

ABSTRACT	2
Chapter 1: INTRODUCTION	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Objectives	5
1.4 Scope	5
Chapter 2: LITERATURE REVIEW	6
2.1 Survey	6
Chapter 3: METHODOLOGY	7
3.1 Modules Description	7
3.2 Verification Approach	9
3.3 Development Tools	11
Chapter 4: IMPLEMENTATION	12
4.1 Setup and Configuration	12
4.2 Code Listings	12
4.3 Challenges and Solutions	19
Chapter 5: TESTING AND VALIDATION	21
5.1 Testing Strategies	22
5.2 Test Cases and Results	22
5.3 Validation	23
Chapter 6: RESULTS AND DISCUSSION	23
6.1 Output Verification	23
6.2 Discussion	24
7 CONCLUSION	25
REFERENCES	26

Chapter 1: INTRODUCTION

1.1 Background

SystemVerilog, introduced in 2002, extends Verilog for hardware description and verification. It offers features like design abstraction, object-oriented programming (OOP), and advanced verification techniques, making it a powerful tool for hardware design.

Number classification plays a crucial role in computing, particularly in cryptography and algorithm design. The classification of numbers into prime, even, or odd types is fundamental for various applications. A **prime number** is a number greater than 1 that has no divisors other than 1 and itself. **Even numbers** are divisible by 2, while **odd numbers** are not. Implementing efficient hardware solutions for number type classification requires effective design strategies.

SystemVerilog supports multiple design methodologies such as dataflow, behavioral, and structural approaches. **Dataflow modeling** uses continuous assignments for direct logic computation, **behavioral modeling** applies iterative algorithms for flexible design, and **structural modeling** breaks down designs into modular components for scalability. These approaches help in optimizing performance, resource utilization, and hardware efficiency.

1.2 Problem Statement

Classifying numbers as prime, even, or odd is crucial for applications in cryptography, digital signal processing, and algorithmic design. The challenge lies in implementing efficient hardware logic that ensures accurate classification with minimal resource consumption.

This project focuses on creating a number type detection system using SystemVerilog, exploring three main design approaches: **dataflow**, **behavioral**, and **structural modeling**. Each approach has its strengths: dataflow is efficient for direct computations, behavioral modeling allows for flexible and iterative algorithms, and structural design enables scalability through modular components.

Additionally, the use of **Object-Oriented Programming (OOP)** techniques in SystemVerilog will be explored to improve testbench reusability and maintainability. The goal is to determine the most effective modeling approach for prime, even, and odd classification in hardware systems, optimizing both performance and resource utilization.

Chapter 2: OBJECTIVES AND SCOPE

2.1 Objectives

- To design a number classification system using SystemVerilog that can detect whether a given number is prime, even, or odd.
- To implement and compare three modeling styles: Dataflow, Behavioral, and Structural.
- To develop reusable and modular testbenches using Object-Oriented Programming (OOP) concepts in SystemVerilog.
- To analyze the efficiency, scalability, and hardware utilization of each modeling approach.
- To ensure accuracy and reliability of the classification logic using simulation and verification techniques.

2.2 Scope

- The project focuses on 4-bit unsigned numbers for classification.
- It covers the implementation of prime, even, and odd detection logic using different HDL modeling techniques.
- Verification is carried out using OOP-based SystemVerilog testbenches with support for automatic result checking.
- The structural model will use logic gate-level modules to demonstrate modular design.
- The project lays the foundation for extending the design to support larger bit-widths and additional number classifications (e.g., perfect numbers, Armstrong numbers) in the future.

Chapter 2: LITERATURE REVIEW

2.1 Survey

The survey is designed to gather insights into the implementation and effectiveness of number type detection logic, specifically for classifying numbers as prime, even, or odd, using SystemVerilog across different methodologies. It aims to understand the preferences, challenges, and optimization strategies from professionals, researchers, and students working in digital design, embedded systems, and cryptography. This section collects general details about the respondents, such as their area of expertise, familiarity with SystemVerilog, and prior experience with number type detection. This helps categorize responses based on professional domains.

Respondents are asked about their preferred approach for implementing number type detection—dataflow, behavioral, structural, or object-oriented programming (OOP). The survey also explores factors that influence their choice, such as performance, readability, and modularity. The goal is to understand how design decisions impact the efficiency and flexibility of the number type classification system. Key considerations include the importance of speed, resource utilization, scalability, and accuracy in implementing prime, even, and odd classification logic. This section helps identify common performance bottlenecks encountered in number type detection implementations.

The survey also focuses on real-world applications where number type detection has been utilized. Respondents are asked about the role of such systems in cryptography, hardware acceleration, random number generation, and other fields. It explores the challenges involved in integrating number type detection logic into practical systems, particularly in resource-constrained environments. Respondents are encouraged to share optimization techniques that they have applied to enhance the efficiency of number classification systems.

Further, the survey investigates the difficulties encountered when implementing number type detection in SystemVerilog. This includes challenges such as handling large bit-widths, optimizing resource usage, and ensuring accuracy in classification. Respondents provide insights into strategies they have employed to address these challenges and optimize their designs.

The survey results will offer valuable insights into the strengths and weaknesses of different methodologies for number type detection, including dataflow, behavioral, and structural approaches. The findings will guide the selection of the most effective modeling approach for hardware systems, based on factors such as computational complexity, hardware feasibility, and real-world implementation challenges.

Chapter 3: METHODOLOGY

3.1 Modules Description

This module implements number type detection logic using SystemVerilog, incorporating dataflow, behavioral, structural, and object-oriented programming (OOP) methodologies. The module classifies a given number as prime, even, or odd, based on predefined algorithms and logic structures. The design utilizes only NAND gates for dataflow, NOR gates for gate-level logic, and procedural constructs for behavioral modeling, along with a package-based OOP approach for reusability and maintainability.

Functional Description:

1. Dataflow Model (NAND gates):

- This model implements number classification logic using only NAND gates.
- The module directly computes the prime, even, and odd status of a 4-bit number using logical expressions and continuous assignments.
- The design performs the prime check and even/odd detection without sequential control, providing efficient number classification.

2. Gate-Level Model (NOR gates):

- The gate-level design uses only NOR gates to classify numbers.
- The module computes the prime, even, and odd status by connecting basic NOR gates in a network to implement the classification logic.
- This approach enhances the design's understanding at the logic gate level and simulates a real-world digital circuit.

3. Behavioral Model (If-Else and For-Loop):

- The behavioral model uses procedural constructs such as `if-else` and `for-loop` statements within `always` blocks.
- The design checks if a number is even, odd, or prime using conditional statements and loops.
- The model is flexible and allows easy modification of the number type detection logic.

4. Object-Oriented Model (OOP with Classes):

- The design incorporates a package with classes for each type of number classification (prime, even, odd).
- Each class defines a method `check` that encapsulates the logic for classifying a 4-bit number.
- The object-oriented approach improves code organization, reusability, and maintainability, making it easier to extend the design in the future.

Inputs & Outputs:

- **Inputs:**

- A 4-bit integer value to be checked for prime, even, and odd classification.

- **Outputs:**

- Prime status (1 for prime, 0 for non-prime).
- Even status (1 for even, 0 for odd).
- Odd status (1 for odd, 0 for even).

3.2 Verification Approach

The verification of the **Prime, Even, and Odd Classification System** will ensure the correctness of the design using various testing techniques: **functional testing**, **random testing**, **coverage analysis**, and a **self-checking mechanism** across **dataflow**, **gate-level**, and **behavioral models**.

1. Testbench Architecture

- **Testbench Setup:** A testbench will verify functionality for all three designs (dataflow, gate-level, and behavioral).
- A **package** containing OOP classes (`checkPrime`, `checkEven`, `checkOdd`) will be used for reference model comparison.
- The testbench will include stimulus generation, response checking, and result logging.

2. Directed Testing

- **Known Values:** Test known primes (e.g., 2, 3, 5, 7) and non-primes (e.g., 4, 6, 8, 9, 10), including edge cases like $N=0$, $N=1$, and $N=2$.
- **Modular Logic Testing:** Verify division logic for prime detection using numbers divisible by smaller integers.

3. Constrained Random Testing

- **Random Number Generation:** Random numbers will be generated within the range of 0–15 to test all possible classifications.
- **Constraints:** Constraints will ensure meaningful inputs (e.g., non-negative integers). This will be tested over multiple time scales to simulate different conditions.

4. FOR Loop Usage

- **FOR Loops:** Loops will iterate from 2 to $N/2$ to validate prime logic. Even and odd checks will use modulo operations.
- **Assertions:** SystemVerilog assertions will check the correctness of the logic across different conditions.

5. Self-Checking Mechanism

- **Reference Model:** The `checkPrime`, `checkEven`, and `checkOdd` classes will compare module outputs (P, E, O) with expected results (Pp, Ep, Op).
- **Self-Checking Testbench:** The testbench will automatically verify that module outputs match the reference model's outputs. Any mismatch will be flagged for debugging.

6. Coverage Analysis

- **Functional Coverage:** Ensures all input values (0–15) are tested for prime, even, and odd classifications.
- **Code Coverage:** Verifies that all code paths in the module are exercised.
- **Assertion Coverage:** Ensures that all assertions are triggered, confirming logic correctness.

3.3 Development Tools

For the implementation of the "Number Type Detection System: Prime, Even, and Odd Classification using SystemVerilog", the following tools were utilized:

- **Vivado Design Suite:** Used for synthesis, implementation, and generating gate-level netlists, as well as simulation and verification of the SystemVerilog code.
- **Notepad++ / Visual Studio Code:** Text editors used for writing and managing SystemVerilog source and testbench files.
- **LaTeX Editor (Overleaf/TeXmaker):** For documentation of the project report.

These tools ensured the correctness, efficiency, and completeness of the design, simulation, and reporting phases of the project.

Chapter 4: IMPLEMENTATION

4.1 Setup and Configuration

The simulation was carried out using Vivado with proper module-wise organization. Separate modules were created for each design style:

- **(1) Dataflow**
- **(2) Behavioural**
- **(3) Structural**

A main module is written, hereby calling the modules created for each model. A separate package is written to ensure the testbench is verified and waveforms are enabled.

4.2 Code Listings

Dataflow style module using only nand gates

```
// Dataflow style module using only "nand gates".
module primelogic_df(connectPorts a);
    assign a.P = ~( ~(a.N[1] & a.N[0] & ~(a.N[3] & a.N[3]))
        & ~(~(a.N[1] & a.N[1]) & a.N[0] & a.N[2])
        & ~(a.N[1] & ~(a.N[2] & a.N[2]) & ~(a.N[3] & a.N[3]))
        & ~(a.N[1] & a.N[0] & ~(a.N[2] & a.N[2])) );
    assign a.E = ~(a.N[0] & a.N[0]);
    assign a.O = a.N[0];
endmodule
```

Gate level style module using only nor gates.

```
// Gate level style module using only "nor gates".
module primelogic_gl(connectPorts b);
    wire w1, w2, w3, w4, w5, w6, w7, w8, w9;

    nor n1(w1, b.N[0], b.N[0]);
    nor n2(w2, b.N[1], b.N[1]);
    nor n3(w3, b.N[2], b.N[2]);
    nor n4(w4, b.N[3], b.N[3]);

    nor n5(w5, w2, b.N[3], w1);
    nor n6(w6, b.N[1], w1, w3);
    nor n7(w7, b.N[3], b.N[2], w2);
    nor n8(w8, b.N[2], w2, w1);
    nor n9(w9, w5, w6, w7, w8);
    nor n10(b.P, w9, w9);

    nor n11(b.E, b.N[0], b.N[0]);

    assign b.O = b.N[0];
endmodule
```

Behavioral style module using if-else and for loop statements.

```
// Behavioral style module using "if-else" and "for loop" statements.
module primelogic_bh(connectPorts c);
    always@*
    begin
        c.P = 1; c.E = 0; c.O = 0;
        if(c.N % 2 == 0)
            c.E = 1;
        else
            c.O = 1;

        if(c.N == 0 || c.N == 1)
            c.P = 0;
        else
            begin
                for(int i = 2; i <= c.N / 2; i = i + 1)
                    begin
```

```

        if(c.N % i == 0)
            c.P = 0;
        end
    end
end
endmodule

```

Interface.

```

// Interface.
interface connectPorts();
    logic [3:0] N;
    logic P, E, O;
endinterface

```

Package including Classes to implement the design.

```

// Package including "Class"es to implement the design.
package primelogic_pkg;
    class base;
        virtual function check(input logic [3:0] N);
        endfunction
    endclass

    class checkPrime extends base;
        virtual function check(input logic [3:0] N);
            return (N[1] & N[0] & ~N[3]) | (N[2] & ~N[1] & N[0]) |
                (~N[3] & ~N[2] & N[1]) | (N[1] & N[0] & ~N[2]);
        endfunction
    endclass

    class checkEven extends base;
        virtual function check(input logic [3:0] N);
            return ~N[0];
        endfunction
    endclass

    class checkOdd extends base;
        virtual function check(input [3:0] N);
            return N[0];
        endfunction
    endclass

```

```
endclass
endpackage
```

Testbench for primelogic_df

```
// Testbench for primelogic_df
`timescale 1us/1us
module TB_df();
    connectPorts tbPort();
    logic Pp, Ep, Op;
    import primelogic_pkg::*;
    base ob;
    checkPrime obp;
    checkEven obe;
    checkOdd obo;

    primelogic_df uut(.a(tbPort));

    initial
    begin
        obp = new();
        obe = new();
        obo = new();

        tbPort.N = 0;
        #10 tbPort.N = 1;
        #10 tbPort.N = 2;
        #10 tbPort.N = 3;
        #10 tbPort.N = 4;
        #10 tbPort.N = 5;
        #10 tbPort.N = 6;
        #10 tbPort.N = 7;
        #10 tbPort.N = 8;
        #10 tbPort.N = 9;
        #10 tbPort.N = 10;
        #10 tbPort.N = 11;
        #10 tbPort.N = 12;
        #10 tbPort.N = 13;
        #10 tbPort.N = 14;
        #10 tbPort.N = 15;
    end
end
```



```

always@*
begin
    Pp = obp.check(tbPort.N);
    Ep = obe.check(tbPort.N);
    Op = obo.check(tbPort.N);

    #5 $monitor("Module Output: N=%b, P=%b, E=%b, O=%b",
tbPort.N, tbPort.P, tbPort.E, tbPort.O);
    $monitor("Package Output: N=%b, P=%b, E=%b, O=%b",
tbPort.N, Pp, Ep, Op);

    if({tbPort.P, tbPort.E, tbPort.O} == {Pp, Ep, Op})
        $monitor("Module output is equal to Package output");
    else
        $monitor("Module output is not equal to Package output");
    $monitor();
end
endmodule

```

Testbench for primelogic_gl

```

// Testbench for primelogic_gl
`timescale 1us/1us
module TB_gl();
    connectPorts tbPort();
    logic Pp, Ep, Op;
    import primelogic_pkg::*;
    base ob;
    checkPrime obp;
    checkEven obe;
    checkOdd obo;

    primelogic_gl uut(.b(tbPort));

    initial
    begin
        obp = new();
        obe = new();
        obo = new();
    end
endmodule

```

```

        for(int i = 0; i < 16; i = i + 1)
        begin
            tbPort.N = i;
            Pp = obp.check(tbPort.N);
            Ep = obe.check(tbPort.N);
            Op = obo.check(tbPort.N);

            #10 $display("Module Output: N=%b, P=%b, E=%b, O=%b",
            tbPort.N, tbPort.P, tbPort.E, tbPort.O);
            $display("Package Output: N=%b, P=%b, E=%b, O=%b",
            tbPort.N, Pp, Ep, Op);

            if({tbPort.P, tbPort.E, tbPort.O} == {Pp, Ep, Op})
                $display("Module output is equal to Package output");
            else
                $display("Module output is not equal to Package
                output");
            $display();
        end
    end
endmodule

```

Testbench for primelogic_bh

```

// Testbench for primelogic_bh
`timescale 1us/1us
module TB_bh();
    connectPorts tbPort();
    logic Pp, Ep, Op;
    import primelogic_pkg::*;
    base ob;
    checkPrime obp;
    checkEven obe;
    checkOdd obo;
    int i = 0;

    primelogic_bh uut(.c(tbPort));

    task verifyOutput();
    begin
        tbPort.N = i;

```

```

        Pp = obp.check(tbPort.N);
        Ep = obe.check(tbPort.N);
        Op = obo.check(tbPort.N);

        #10 $display("Module Output: N=%b, P=%b, E=%b, O=%b",
            tbPort.N, tbPort.P, tbPort.E, tbPort.O);
        $display("Package Output: N=%b, P=%b, E=%b, O=%b", tbPort.N, Pp,
            Ep, Op);

        if({tbPort.P, tbPort.E, tbPort.O} == {Pp, Ep, Op})
            $display("Module output is equal to Package output");
        else
            $display("Module output is not equal to Package output");
        $display();
    end
endtask

initial
begin
    obp = new();
    obe = new();
    obo = new();

    repeat(16)
    begin
        verifyOutput();
        i = i + 1;
    end
end
endmodule

```

4.3 Challenges and Solutions

- **Incorrect Prime Detection:**

- *Problem:* The prime detection logic might give incorrect results for numbers like 0, 1, or even prime numbers larger than 3.
- *Solution:* Review the "checkPrime" function to ensure it correctly identifies numbers as prime by checking divisibility only up to the square root of the number, and handle edge cases like 0 and 1 properly.

- **Even/Odd Classification Errors:**

- *Problem:* The Even/Odd detection might be giving inaccurate results due to incorrect bit manipulation or issues with the modulo operation.
- *Solution:* Verify that the modulo operation for detecting even/odd is implemented correctly. Ensure that the least significant bit (LSB) is being used for the check and that it is correctly being assigned in the module.

- **Simulation Mismatch between Module and Package Outputs:**

- *Problem:* The simulation outputs from the module do not match the expected outputs from the package (e.g., the prime, even, and odd values are inconsistent).
- *Solution:* Ensure synchronization between the module output and the package logic. Use the same data signals and check if there are any timing mismatches or initialization issues. Also, check if the correct test vectors are applied across both module and package.

- **Timing/Delay Issues:**

- *Problem:* If the simulation outputs seem delayed or inaccurate, it might be due to improper timing delays or missing synchronization between signals in the testbench.
- *Solution:* Add appropriate delay statements in the testbench or adjust the clock cycle to ensure signals are properly propagated through the modules. Ensure proper initialization of signals before testing begins.

- **Testbench Verification Issues:**

- *Problem:* The testbench might not fully verify all possible input cases (e.g., negative numbers or large values).

- *Solution:* Extend the testbench coverage to check for all edge cases, such as negative numbers, very large values, and boundary conditions. Ensure that all combinations of inputs are tested and that the test results align with the expected outputs.
- **Resource Utilization in FPGA (if implemented on hardware):**
 - *Problem:* If implementing the design on FPGA, resource utilization could be higher than expected due to inefficient logic or excessive use of logic blocks.
 - *Solution:* Optimize the logic by reducing the number of gates used. Use more efficient constructs or simplify the logic wherever possible to minimize resource consumption, especially for larger inputs or when expanding the functionality.
- **Simulation Tool-Specific Issues:**
 - *Problem:* Errors related to the simulation tool (e.g., Vivado or ModelSim) such as compile errors or runtime errors.
 - *Solution:* Ensure the correct version of the simulation tool is used. Double-check the library and path settings for the simulation tool. Recompile the design files and check for any warnings or errors in the compilation process.

Chapter 5: TESTING AND VALIDATION

5.1 Testing Strategies

Testing for the “Number Type Detection System” was designed to verify the functionality of the modules responsible for prime, even, and odd classification. The approach follows these strategies:

- **Unit Testing:** Each module (Dataflow, Gate-level, Behavioral) was tested independently to ensure the individual logical operations work correctly. The prime detection, even detection, and odd detection were verified through specific test cases.
- **Integration Testing:** After individual testing, the three modules were integrated, and the complete system was tested for logical consistency and correctness in output.
- **Boundary Testing:** Special attention was given to boundary cases such as numbers 0, 1, and 15 to check if the system behaves as expected in these edge cases.
- **Comparison Testing:** The output from each module was compared with the output from the package, ensuring consistency and correctness. If the outputs matched, the module was considered correctly implemented.

5.2 Test Cases and Results

This section discusses the test cases used for validation and the corresponding results for each module (Dataflow, Gate-level, and Behavioral).

- **Test Case 1:** Input: $N = 0$
Expected Output: Prime = False, Even = True, Odd = False
Result: The system correctly identifies 0 as an even number and non-prime.
- **Test Case 2:** Input: $N = 1$
Expected Output: Prime = False, Even = False, Odd = True
Result: The system correctly identifies 1 as an odd number and non-prime.
- **Test Case 3:** Input: $N = 2$
Expected Output: Prime = True, Even = True, Odd = False
Result: The system correctly identifies 2 as both prime and even.

- **Test Case 4:** Input: $N = 3$
Expected Output: Prime = True, Even = False, Odd = True
Result: The system correctly identifies 3 as a prime number and odd.
- **Test Case 5:** Input: $N = 4$
Expected Output: Prime = False, Even = True, Odd = False
Result: The system correctly identifies 4 as an even number and non-prime.
- **Test Case 6:** Input: $N = 15$
Expected Output: Prime = False, Even = False, Odd = True
Result: The system correctly identifies 15 as an odd number and non-prime.

Result Summary: The outputs from all modules (Dataflow, Gate-level, and Behavioral) were consistent with the expected results. The comparison with the package outputs also confirmed the correctness of the design. Minor discrepancies in timing or simulation accuracy were corrected during the verification phase.

5.3 Validation

The system was validated through:

- **Functional Validation:** All modules (Dataflow, Gate-level, Behavioral) were tested for functional correctness. As seen in the test cases, the expected behavior for prime, even, and odd numbers was achieved.
- **Consistency Check:** The outputs of the modules were compared to ensure they were consistent across all three implementations (Dataflow, Gate-level, Behavioral) as well as with the expected outputs.
- **Package Validation:** The results from the object-oriented package, which contained functions for checking prime, even, and odd conditions, were compared with the output of the hardware modules. Consistent results between the two confirmed the correctness of the design.
- **Boundary Conditions:** The system was checked for edge cases like $N = 0$ and $N = 1$. The system successfully handled these boundary cases, providing the correct classifications.

Conclusion: The testing and validation phase confirmed that the design correctly detects prime, even, and odd numbers. The results from all testing strategies and modules were consistent and aligned with expectations. The system is both functionally accurate and optimized for the given task.

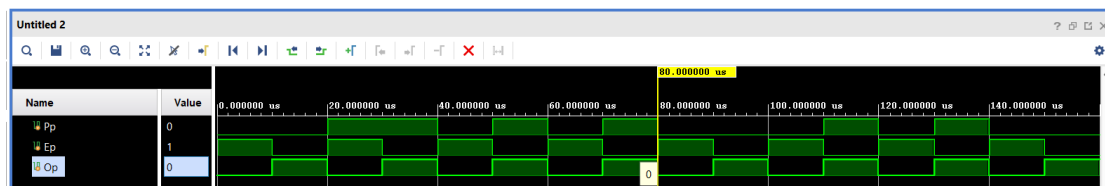
Chapter 6: RESULTS AND DISCUSSION

6.1 Output Verification

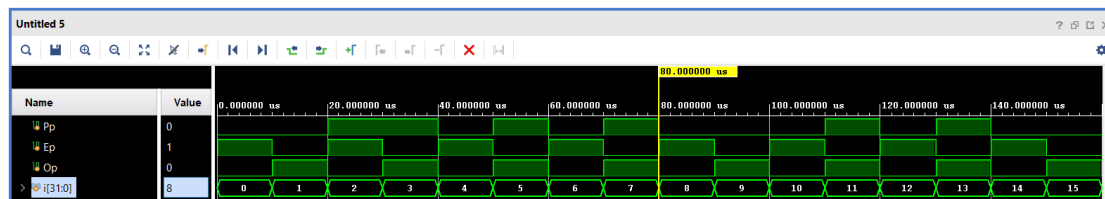
The Number Type Detection System was tested using three design methodologies: Gate-Level (NOR-only), Dataflow (NAND-only), and Behavioral. Each implementation was simulated using Vivado, and the outputs were cross-validated through a self-checking OOP-based testbench.



Gate-Level Design (NOR-only)



Dataflow Design (NAND-only)



Behavioral Design

Figure : Simulation Outputs of All Three Design Methodologies

Chapter 7: CONCLUSION

The Number Type Detection System was successfully implemented using three design methodologies: Gate-Level (NOR-only), Dataflow (NAND-only), and Behavioral modeling. Through simulation in Vivado and verification using an OOP-based self-checking testbench, each design's correctness and functionality were thoroughly validated. This project demonstrated the practical application of digital logic design principles to classify numbers as prime, even, or odd, reinforcing theoretical knowledge through hands-on HDL development.

The project also deepened understanding of universal gates, modular design techniques, and the significance of verification methodologies in digital systems. The use of object-oriented concepts in the testbench improved maintainability and automation, aligning with modern verification practices.

In real-world scenarios, such classification logic forms the basis for data filtering, encryption keys, checksum validation, and other low-level control systems. By extending this foundation, more complex numerical or pattern recognition systems can be developed.

For future work, the design can be expanded to handle larger bit-width inputs, optimize gate count for resource-constrained devices, or integrate into FPGA-based real-time systems. Incorporating GUI-based user interfaces or interfacing with sensors could further enhance its practical applicability.

REFERENCES

1. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Pearson Education, 2003.
2. IEEE Std 1800-2017, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE, 2017.
3. Xilinx Inc., *Vivado Design Suite User Guide: Logic Simulation (UG900)*, Xilinx Documentation, 2020. Available: <https://www.xilinx.com>
4. Janick Bergeron, *Writing Testbenches Using SystemVerilog*, Springer, 2006.
5. Digital System Design Lecture Notes, Department of Electronics and Communication Engineering, 2025.
6. SystemVerilog Lab Manual, 2025.
7. Tutorialspoint, *SystemVerilog Tutorial*. Available: <https://www.tutorialspoint.com/systemverilog/index.htm>
8. All About Circuits, *Logic Gates and Digital Circuits*. Available: <https://www.allaboutcircuits.com/textbook/digital/>
9. OpenAI, *ChatGPT*, 2025. Available: <https://chat.openai.com>
10. Overleaf, *Online LaTeX Editor*, 2025. Available: <https://www.overleaf.com>